Guía paso a paso · desde cero

Conectar React con una API

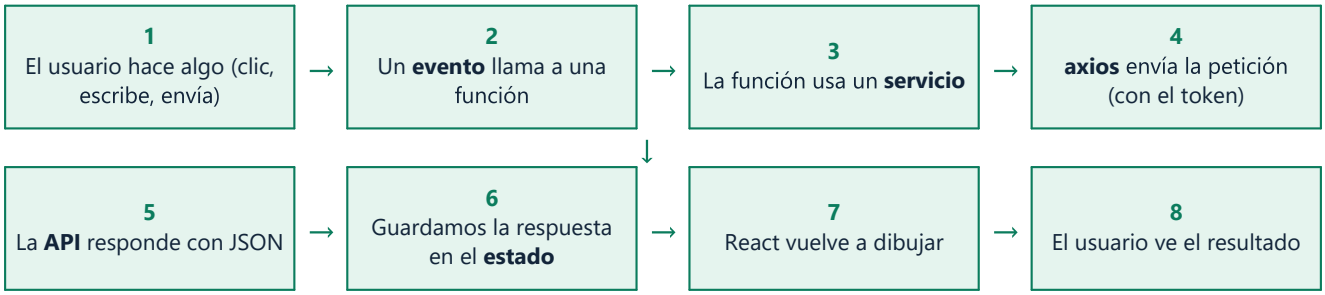
Vamos a crear una app nueva, archivo por archivo: conexión con el backend, eventos, rutas, login, token y rutas protegidas

¿Qué vamos a construir?

Una mini tienda con 3 pantallas, conectada a la API de clase (`https://api-funval-g6.onrender.com`):

Pantalla	URL	Qué hace
Login	`/login`	El usuario escribe su correo y contraseña. Si son correctos, entra.
Productos	`/productos`	Lista de productos traída de la API, con un buscador. **Solo con sesión.**
Detalle	`/productos/15`	Muestra un solo producto. **Solo con sesión.**

Cómo viaja la información



Los archivos que vamos a crear

```
src/
├── types/producto.ts          Parte 1: como es un producto
├── lib/axios.ts               Parte 1: la conexion con la API (Parte 4: el token)
├── services/productos.ts      Parte 1: pedir productos
├── hooks/useApi.ts            Parte 1: no repetir codigo de carga
├── components/Buscador.tsx    Parte 2: evento entre componentes
├── pages/ProductosPage.tsx    Parte 1 y 2
├── pages/LoginPage.tsx        Parte 2: formulario (Parte 4: login real)
├── main.tsx                   Parte 3: activar el router
├── App.tsx                    Parte 3: el mapa de rutas
├── components/Layout.tsx      Parte 3: header + menu
├── pages/ProductoDetallePage.tsx   Parte 3: useParams
├── pages/NoEncontradaPage.tsx      Parte 3: pagina 404
├── services/auth.ts           Parte 4: login
├── store/useAuthStore.ts      Parte 4: guardar el token
└── components/RutaProtegida.tsx    Parte 4: bloquear paginas
```

PARTE 0

Preparar el proyecto

Antes de escribir código necesitamos un proyecto de React vacío y las librerías que vamos a usar.

Paso 1 · Crear el proyecto con Vite

```
terminal
npm create vite@latest mi-tienda -- --template react-ts
cd mi-tienda
npm install
```

Esto crea una carpeta `mi-tienda` con React + TypeScript listo para usar.

Paso 2 · Instalar las librerías

```
terminal
npm install axios react-router-dom zustand
```

Librería	Para qué la usamos
`axios`	Hablar con la API (enviar y recibir datos)
`react-router-dom`	Tener varias páginas con URLs distintas
`zustand`	Guardar la sesión (el token) y compartirla con toda la app

Paso 3 · Limpiar lo que trae Vite

- Borra el contenido de `src/App.css` y `src/index.css` (o deja `index.css` con tus estilos).
- Borra la carpeta `src/assets` si no la vas a usar.
- Crea las carpetas vacías: `types`, `lib`, `services`, `hooks`, `store`, `components`, `pages` dentro de `src`.

Paso 4 · Mirar la API antes de programar

Abre en el navegador **https://api-funval-g6.onrender.com/products/**. Verás el JSON que devuelve la API:

```
[
  {
    "id": 34,
    "name": "Galleta con chispas",
    "price": "8.00",
    "category": "Panaderia-galleta",
    "image_url": "https://res.cloudinary.com/..."
  },
  ...
]
```

Y en **https://api-funval-g6.onrender.com/docs** está la documentación (Swagger) con todas las rutas.

> **Siempre empieza por aquí:** antes de escribir el servicio, mira **qué campos** manda la API y **cómo se llaman**. La mitad de los errores son por usar un nombre distinto (`nombre` en vez de `name`).

PARTE 1

Conexión con el backend

El frontend (React) dibuja la pantalla. El backend (la API) guarda los datos. Para comunicarse, el frontend envía **peticiones** y la API devuelve **respuestas**.

> **Analogía:** un restaurante. Tú (el frontend) no entras a la cocina: le haces un **pedido** al mesero (la API), el mesero va a la cocina (la base de datos) y te trae el plato (la respuesta en JSON). Si pides algo que no existe, el mesero te dice que no (un error).

Los 4 tipos de pedido (métodos HTTP)

Método	Para qué sirve	Ejemplo
`GET`	Leer datos	`GET /products/` → la lista de productos
`POST`	Crear o enviar algo	`POST /auth/login` → enviar correo y contraseña
`PUT`	Actualizar	`PUT /auth/perfil` → cambiar nombre y foto
`DELETE`	Borrar	`DELETE /products/5` → borrar el producto 5

Paso 5 · Definir cómo es un producto (el tipo)

Primero le decimos a TypeScript quE forma tiene un producto **en nuestra app**. Usamos nombres en español:

```
src/types/producto.ts · NUEVO
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  categoria: string;
  imagenUrl: string | null;
}
```

Paso 6 · Crear la conexión con la API

En vez de escribir la dirección completa en cada pedido, creamos **una sola conexión** con la dirección base. Por ahora el archivo es así de corto (en la Parte 4 le agregamos el token):

```
src/lib/axios.ts · NUEVO
import axios from "axios";

export const api = axios.create({
  baseURL: "https://api-funval-g6.onrender.com",
});
```

Esta parte	Qué hace
`axios.create({...})`	Crea una conexión con configuración propia
`baseURL`	La parte de la dirección que se repite. Después solo escribimos `"/products/"`
`export const api`	La exportamos para usarla en los servicios

Paso 7 · Crear el servicio de productos

Un **servicio** reúne todas las funciones que piden datos de un mismo tema. Los componentes no van a saber nada de URLs: solo llamarán a `productosService.getTodos()`.

```
src/services/productos.ts · NUEVO
import { api } from "../lib/axios";
import type { Producto } from "../types/producto";

// asi llega un producto desde la API
interface ProductoApi {
  id: number;
  name: string;
  price: string;
  category: string | null;
  image_url: string | null;
}

// lo pasamos a nuestro formato
function convertir(p: ProductoApi): Producto {
  return {
    id: p.id,
    nombre: p.name,
    precio: Number(p.price),
    categoria: p.category ?? "Sin categoría",
    imagenUrl: p.image_url,
  };
}

export const productosService = {
  getTodos: async (): Promise<Producto[]> => {
    const respuesta = await api.get<ProductoApi[]>("/products/");
    return respuesta.data.map(convertir);
  },
};
```

Esta parte	Qué hace
`interface ProductoApi`	Cómo llega el producto **desde la API** (en ingl s, el precio como texto)
`function convertir`	Lo transforma a **nuestro** tipo `Producto` (español, precio como número)
`async` / `await`	Pedir datos tarda. `await` espera la respuesta antes de seguir
`api.get<ProductoApi[]>(...)`	Hace un `GET`. Lo de `< >` le dice a TypeScript qu va a llegar
`respuesta.data`	Axios pone el JSON de la respuesta dentro de `.data`
`.map(convertir)`	Convierte cada producto de la lista

Paso 8 · Mostrar los productos en una página (versión 1)

Ahora usamos el servicio en un componente. Toda pantalla que trae datos tiene que manejar **3 situaciones**: está cargando, falló o ya llegaron los datos.

```
src/pages/ProductosPage.tsx · NUEVO (versión 1)
import { useEffect, useState } from "react";
import { productosService } from "../services/productos";
import type { Producto } from "../types/producto";

export function ProductosPage() {
  const [productos, setProductos] = useState<Producto[]>([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const cargar = async () => {
      try {
        const datos = await productosService.getTodos();
        setProductos(datos);
      } catch {
        setError("No pudimos cargar los productos");
      } finally {
        setCargando(false);
      }
    };

    cargar();
  }, []);

  if (cargando) return <p>Cargando...</p>;
  if (error) return <p>{error}</p>;

  return (
    <ul>
      {productos.map((p) => (
        <li key={p.id}>{p.nombre} - Bs {p.precio}</li>
      ))}
    </ul>
  );
}
```

Esta parte	Qué hace
3 `useState`	Los datos, si está cargando y el mensaje de error
`useEffect(() => {...}, [])`	Ejecuta el código **una sola vez**, cuando la página aparece. El `[]` vacío significa "solo al inicio"
`const cargar = async () => {}`	`useEffect` no puede ser `async`, por eso creamos una función adentro y la llamamos
`try`	Intenta pedir los datos
`catch`	Si algo falla (sin internet, error de la API), guarda un mensaje
`finally`	Pase lo que pase, deja de mostrar "Cargando..."
`if (cargando) return ...`	Según el estado, mostramos una cosa u otra

Para verla, pon `<ProductosPage />` dentro de `App.tsx` y corre `npm run dev`. **Ya estás conectado al backend.**

Paso 9 · El problema: repetir lo mismo en cada página

Mira de nuevo la versión 1: los 3 `useState`, el `useEffect`, el `try / catch / finally`... En la página de detalle, en el perfil, en cualquier lista, vamos a necesitar **exactamente lo mismo**. Solo cambia **qué función** llamamos.

> **Solución:** sacar ese código repetido a un **hook propio**. Un hook es una función que empieza con `use` y puede usar otros hooks (`useState`, `useEffect`). Le pasamos la función que pide los datos y nos devuelve todo listo: `data`, `cargando`, `error` y `recargar`.

Paso 10 · Crear el hook useApi

```
src/hooks/useApi.ts · NUEVO
import { useEffect, useState } from "react";

export function useApi<T>(
  pedir: () => Promise<T>,
  dependencias: unknown[] = [],
) {
  const [data, setData] = useState<T | null>(null);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [intento, setIntento] = useState(0);

  useEffect(() => {
    let cancelado = false;
    setCargando(true);
    setError(null);

    pedir()
      .then((resultado) => {
        if (!cancelado) setData(resultado);
      })
      .catch(() => {
        if (!cancelado) setError("No pudimos cargar los datos");
      })
      .finally(() => {
        if (!cancelado) setCargando(false);
      });

    return () => {
      cancelado = true;
    };
  }, [intento, ...dependencias]);

  const recargar = () => setIntento((n) => n + 1);

  return { data, cargando, error, recargar };
}
```

Esta parte	Qué hace
`useApi<T>(...)`	`T` es "cualquier tipo". Si pides productos, `data` será `Producto[]`; si pides uno, será `Producto`
`pedir: () => Promise<T>`	Recibe **la función** que pide los datos (no los datos)
`dependencias`	Si alguna cambia, vuelve a pedir (por ejemplo, el `id` del producto)
`intento` + `recargar`	Para el botón **Reintentar**: sumar 1 a `intento` vuelve a ejecutar el `useEffect`

Esta parte	Qué hace
`.then / .catch / .finally`	Lo mismo que `try / catch / finally`, escrito con promesas
`cancelado`	Si el usuario se va de la página antes de que llegue la respuesta, no intentamos guardar nada
`return { ... }`	Lo que puede usar cualquier página

Paso 11 · Usar useApi en la página de productos (versión 2)

src/pages/ProductosPage.tsx · ACTUALIZAR

```
import { useApi } from "../hooks/useApi";
import { productosService } from "../services/productos";

export function ProductosPage() {
  const { data, cargando, error, recargar } = useApi(() =>
    productosService.getTodos(),
  );

  if (cargando) return <p>Cargando...</p>;

  if (error) {
    return (
      <div>
        <p>{error}</p>
        <button onClick={recargar}>Reintentar</button>
      </div>
    );
  }

  return (
    <ul>
      {(data ?? []).map((p) => (
        <li key={p.id}>{p.nombre} - Bs {p.precio}</li>
      ))}
    </ul>
  );
}
```

Versión 1 (sin useApi)	Versión 2 (con useApi)
3 `useState` + `useEffect` + `try/catch`	1 línea: `useApi(() => productosService.getTodos())`
Hay que copiarlo en cada página	Se reutiliza en todas las páginas
No tenía botón para reintentar	`recargar` viene incluido

> `data ?? []`: mientras no llegan los datos, `data` vale `null`. `??` significa "si es null, usa esto otro", así `.map` nunca falla.

PARTE 2

Eventos

Un evento es **algo que hace el usuario**: un clic, escribir una letra, enviar un formulario. A cada elemento le pasamos una función que React ejecuta cuando ese evento ocurre.

Evento	Cuándo ocurre	En nuestra app
`onClick`	Al hacer clic	Botón **Reintentar**, botón **Cerrar sesión**
`onChange`	Cada vez que cambia un input	El buscador, los campos del login
`onSubmit`	Al enviar un formulario (botón o Enter)	El formulario de login

onClick: ya lo usamos

En la versión 2 de productos ya hay un evento: `<button onClick={recargar}>Reintentar</button>`.

> **Sin paréntesis:** `onClick={recargar}` le **entrega** la función a React para que la llame al hacer clic. `onClick={recargar()}` la **ejecuta de inmediato** al dibujar. Si necesitas pasar un dato: `onClick={() => borrar(5)}`.

Paso 12 · onChange: un buscador

Vamos a filtrar los productos mientras el usuario escribe. Primero un componente `Buscador`:

```
src/components/Buscador.tsx · NUEVO
interface Props {
  valor: string;
  onCambiar: (nuevo: string) => void;
}

export function Buscador({ valor, onCambiar }: Props) {
  return (
    <input
      value={valor}
      onChange={(e) => onCambiar(e.target.value)}
      placeholder="Buscar producto..."
    />
  );
}
```

Esta parte	Qué hace
`valor`	Lo que está escrito ahora (lo decide el padre)
`onCambiar`	Una **función** que el padre nos presta para avisarle que el usuario escribió algo
`onChange={(e) => ...}`	Cada letra que se escribe dispara este evento
`e.target.value`	El texto que hay en el input en ese momento

> **El hijo avisa, el padre decide:** `Buscador` no guarda la búsqueda. Solo **avisa** con `onCambiar`. Quien guarda y usa el valor es la página. Por eso las props que son funciones se llaman `on...`.

Paso 13 · Usar el buscador en la página de productos (versión 3)

src/pages/ProductosPage.tsx · ACTUALIZAR (versión final)

```tsx
import { useState } from "react";
import { Link } from "react-router-dom";
import { Buscador } from "../components/Buscador";
import { useApi } from "../hooks/useApi";
import { productosService } from "../services/productos";

export function ProductosPage() {
  const { data, cargando, error, recargar } = useApi(() =>
    productosService.getTodos(),
  );
  const [busqueda, setBusqueda] = useState("");

  if (cargando) return <p>Cargando...</p>;

  if (error) {
    return (
      <div>
        <p>{error}</p>
        <button onClick={recargar}>Reintentar</button>
      </div>
    );
  }

  const productos = (data ?? []).filter((p) =>
    p.nombre.toLowerCase().includes(busqueda.toLowerCase()),
  );

  return (
    <section>
      <h1>Productos</h1>
      <Buscador valor={busqueda} onCambiar={setBusqueda} />

      {productos.length === 0 && <p>No hay productos con ese nombre</p>}

      <ul>
        {productos.map((p) => (
          <li key={p.id}>
            <Link to={`/productos/${p.id}`}>{p.nombre}</Link> - Bs {p.precio}
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Esta parte	Qué hace
`useState("")`	Guarda lo que el usuario escribió en el buscador
`onCambiar={setBusqueda}`	Cuando el Buscador avisa, actualizamos el estado y React vuelve a dibujar
`.filter(...)`	Deja solo los productos cuyo nombre incluye el texto buscado
`.toLowerCase()`	Para que "PAN" y "pan" sean iguales
`<Link to=...>`	Un enlace al detalle (lo armamos en la Parte 3)

> **Los hooks van arriba:** `useApi` y `useState` están **antes** de los `if (...) return`. Los hooks siempre se llaman en el mismo orden, nunca dentro de un `if`.

Paso 14 · onSubmit: el formulario de login (versión 1)

Por ahora solo mostramos en consola lo que se escribió. En la Parte 4 lo conectamos con la API.

```
src/pages/LoginPage.tsx · NUEVO (versión 1)
import { useState, type FormEvent } from "react";

export function LoginPage() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();               // que el navegador NO recargue la pagina
    console.log("Enviar:", email, password);
  };

  return (
    <form onSubmit={handleSubmit}>
      <h1>Iniciar sesión</h1>
      <label>
        Correo
        <input type="email" value={email}
               onChange={(e) => setEmail(e.target.value)} required />
      </label>
      <label>
        Contraseña
        <input type="password" value={password}
               onChange={(e) => setPassword(e.target.value)} required />
      </label>
      <button type="submit">Ingresar</button>
    </form>
  );
}
```

Esta parte	Qué hace
`value={email}` + `onChange`	Input **controlado**: React siempre sabe lo que hay escrito
`onSubmit={handleSubmit}`	Se ejecuta al apretar el botón o Enter
`e.preventDefault()`	Por defecto el navegador recarga la página al enviar un formulario. Esto lo evita
`type="submit"`	Este botón envía el formulario
`required`	El navegador no deja enviar si el campo está vacío

PARTE 3

Rutas

Nuestra app es una **SPA** (Single Page Application): el navegador carga una sola página y React Router cambia lo que se ve según la URL, **sin recargar**.

Paso 15 · Activar el router

```
src/main.tsx · ACTUALIZAR
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./index.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

`BrowserRouter` envuelve toda la app. Sin él, nada de lo que sigue funciona.

Paso 16 · Crear el layout (header + menú)

Las páginas privadas comparten un header. En vez de copiarlo en cada página, creamos un **Layout**:

```
src/components/Layout.tsx · NUEVO (versión 1)
import { NavLink, Outlet } from "react-router-dom";

export function Layout() {
  return (
    <div>
      <header>
        <strong>Mi tienda</strong>
        <nav>
          <NavLink to="/productos">Productos</NavLink>
        </nav>
      </header>

      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

Esta parte	Qué hace
`<NavLink to=...>`	Un enlace interno. A diferencia de `<a href>`, **no recarga** la app. Además sabe si está activo
`<Outlet />`	Un "hueco": ahí aparece la página que corresponde a la URL

Paso 17 · La página de detalle (useParams)

Para mostrar un solo producto necesitamos su `id`, que viene **en la URL**: `/productos/15`. Primero agregamos la función al servicio:

```
src/services/productos.ts · AGREGAR dentro de productosService
  getPorId: async (id: number): Promise<Producto> => {
    const respuesta = await api.get<ProductoApi>(`/products/${id}`);
    return convertir(respuesta.data);
  },
```

```
src/pages/ProductoDetallePage.tsx · NUEVO
import { Link, useParams } from "react-router-dom";
import { useApi } from "../hooks/useApi";
import { productosService } from "../services/productos";

export function ProductoDetallePage() {
  const { id } = useParams(); // "15" si la URL es /productos/15

  const { data: producto, cargando, error } = useApi(
    () => productosService.getPorId(Number(id)),
    [id],
  );

  if (cargando) return <p>Cargando...</p>;
  if (error || !producto) return <p>No encontramos el producto</p>;

  return (
    <article>
      <Link to="/productos">← Volver</Link>
      <h1>{producto.nombre}</h1>
      {producto.imagenUrl && (
        <img src={producto.imagenUrl} alt={producto.nombre} width={240} />
      )}
      <p>Categoría: {producto.categoria}</p>
      <p>Precio: Bs {producto.precio}</p>
    </article>
  );
}
```

Esta parte	Qué hace
`useParams()`	Lee las partes variables de la URL. Si la ruta es `/productos/:id` y la URL `/productos/15`, `id` vale `"15"`
`Number(id)`	Lo que viene en la URL siempre es **texto**: lo convertimos a número
`[id]`	Si cambia el id (otro producto), `useApi` vuelve a pedir
`producto.imagenUrl && (...)`	Solo muestra la imagen si existe

Paso 18 · Página 404

```
src/pages/NoEncontradaPage.tsx · NUEVO
import { Link } from "react-router-dom";

export function NoEncontradaPage() {
  return (
    <div>
      <h1>404 - Página no encontrada</h1>
      <Link to="/">Volver al inicio</Link>
    </div>
  );
}
```

Paso 19 · El mapa de rutas (versión 1)

```
src/App.tsx · ACTUALIZAR (versión 1)
import { Navigate, Route, Routes } from "react-router-dom";
import { Layout } from "./components/Layout";
import { LoginPage } from "./pages/LoginPage";
import { NoEncontradaPage } from "./pages/NoEncontradaPage";
import { ProductoDetallePage } from "./pages/ProductoDetallePage";
import { ProductosPage } from "./pages/ProductosPage";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/productos" replace />} />
      <Route path="/login" element={<LoginPage />} />

      <Route element={<Layout />}>
        <Route path="/productos" element={<ProductosPage />} />
        <Route path="/productos/:id" element={<ProductoDetallePage />} />
      </Route>

      <Route path="*" element={<NoEncontradaPage />} />
    </Routes>
  );
}

export default App;
```

Esta parte	Qué hace
`<Routes>`	Revisa la URL y elige **una** ruta
`path` + `element`	"Si la URL es esta, muestra este componente"
`<Navigate to=... replace />`	Redirige. Entrar a `/` te lleva a `/productos`
`<Route element={<Layout />}>`	Ruta **sin path** que envuelve a otras: todas sus hijas se dibujan dentro del `<Outlet />` del Layout
`:id`	Parte variable de la URL (la lee `useParams`)
`path="*"`	Cualquier otra URL: la página 404

1 Entro a `/productos/15` → **2** `Routes` busca la ruta que coincide → **3** Dibuja `Layout` (header + menú) → **4** En el `<Outlet />` aparece `ProductoDetallePage`

Moverse entre páginas: resumen

Herramienta	Cuándo
`<Link to="/ruta">`	Un enlace que el usuario hace clic
`<NavLink to="/ruta">`	Igual, pero sabe si está activo (para el menú)
`useNavigate()`	Ir a otra página **desde código**, despu s de algo (lo usamos en el login)
`<Navigate to="/ruta" />`	Redirigir **mientras se dibuja** (lo usamos para proteger rutas)
`useParams()`	Leer `/productos/:id`
`useSearchParams()`	Leer `?pagina=2&buscar=pan` (filtros, paginación)

> **Prueba ahora:** entra a `/productos`, haz clic en un producto, vuelve, escribe una URL inventada (`/hola`). Todo cambia sin recargar la página. Pero **cualquiera** puede entrar a `/productos` sin iniciar sesión: eso lo arreglamos en con la autenticación y el tken

La API responde al login con esto:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user_name": "Ana",
  "user_role": "cliente"
}
```

Esta parte	Qué hace
`login`	Hace `POST /auth/login` enviando `{ email, password }`
`yo`	Hace `GET /auth/me`. Es una ruta **privada**: solo responde si mandamos un token válido. La usamos para comprobar que todo funciona

¿Qué es ese token? (JWT)

Es un **JWT** (JSON Web Token): un texto largo con 3 partes separadas por puntos:

Parte	Qué tiene
Header	El tipo de token
Payload	A quien pertenece (tu email) y **cuándo vence**
Firma	Un sello que solo el backend sabe crear. Si alguien cambia una letra, la API lo rechaza

Paso 21 · Guardar la sesión (store con Zustand)

Necesitamos guardar el token en un lugar al que **toda la app** pueda llegar (el login lo guarda, axios lo lee, el header muestra el nombre). Para eso usamos un **store** de Zustand:

```
src/store/useAuthStore.ts · NUEVO
import { create } from "zustand";
import { persist } from "zustand/middleware";

interface AuthState {
  token: string | null;
  nombre: string | null;
  rol: string | null;
  iniciarSesion: (token: string, nombre: string, rol: string) => void;
  cerrarSesion: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      nombre: null,
      rol: null,

      iniciarSesion: (token, nombre, rol) => set({ token, nombre, rol }),

      cerrarSesion: () => set({ token: null, nombre: null, rol: null }),
    }),
    { name: "sesion" }, // nombre de la clave en localStorage
  ),
);
```

Esta parte	Qué hace
`create<AuthState>()`	Crea el store. `AuthState` dice qu datos y funciones tiene
`token`, `nombre`, `rol`	Los datos de la sesión (`null` = no hay sesión)
`iniciarSesion(...)`	Guarda los datos. `set` actualiza el store y avisa a los componentes
`cerrarSesion()`	Vuelve todo a `null`
`persist(..., { name: "sesion" })`	Copia el store en **localStorage** con la clave `sesion`. Así, si el usuario recarga (F5), la sesión sigue ahí

Paso 22 · Conectar el formulario de login con la API (versión 2)

src/pages/LoginPage.tsx · ACTUALIZAR (versión final)

```tsx
import { useState, type FormEvent } from "react";
import { useNavigate } from "react-router-dom";
import { authService } from "../services/auth";
import { useAuthStore } from "../store/useAuthStore";

export function LoginPage() {
  const navigate = useNavigate();
  const iniciarSesion = useAuthStore((s) => s.iniciarSesion);

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState<string | null>(null);
  const [enviando, setEnviando] = useState(false);

  const handleSubmit = async (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault(); // que el navegador NO recargue la pagina
    setError(null);
    setEnviando(true);

    try {
      const respuesta = await authService.login({ email, password });
      iniciarSesion(
        respuesta.access_token,
        respuesta.user_name,
        respuesta.user_role,
      );
      navigate("/productos", { replace: true });
    } catch {
      setError("Correo o contraseña incorrectos");
    } finally {
      setEnviando(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <h1>Iniciar sesión</h1>

      <label>
        Correo
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
      </label>

      <label>
        Contraseña
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />
      </label>

      {error && <p>{error}</p>}
      <button type="submit" disabled={enviando}>
        {enviando ? "Ingresando..." : "Ingresar"}
      </button>
    </form>
  );
}
```

Esta parte	Qué hace
`useAuthStore((s) => s.iniciarSesion)`	Saca del store la función para guardar la sesión
`async` en `handleSubmit`	Ahora espera la respuesta de la API
`authService.login({ email, password })`	Envía los datos. Si son incorrectos, la API responde error y salta al `catch`
`iniciarSesion(...)`	Guarda token, nombre y rol
`navigate("/productos", { replace: true })`	Lleva a productos. `replace` hace que el botón Atrás no vuelva al login
`enviando`	Desactiva el botón mientras espera, para que no se envíe dos veces

Paso 23 · Enviar el token en cada petición (interceptor)

La API reconoce al usuario por un **header**: `Authorization: Bearer <token>`. Podríamos agregarlo a mano en cada servicio, pero es fácil olvidarlo. Mejor usamos un **interceptor**: una función que axios ejecuta **antes de enviar cada petición**.

src/lib/axios.ts · ACTUALIZAR

```
import axios from "axios";
import { useAuthStore } from "../store/useAuthStore";

export const api = axios.create({
  baseURL: "https://api-funval-g6.onrender.com",
});

// antes de cada peticion: agregar el token
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Esta parte	Qué hace
`interceptors.request.use`	"Antes de que salga cualquier petición, pasa por aquí"
`useAuthStore.getState()`	Lee el store **fuera de un componente**. Los hooks (`useAuthStore(...)`) solo funcionan dentro de componentes
`config.headers.Authorization`	Agrega el header con el token
`return config`	Deja que la petición siga su camino (si lo olvidas, ninguna petición sale)

Paso 24 · Proteger las páginas privadas (guard)

Un **guard** es un componente que se pone **delante** de las rutas privadas y decide si dejar pasar:

src/components/RutaProtegida.tsx · NUEVO

```
import { Navigate, Outlet } from "react-router-dom";
import { useAuthStore } from "../store/useAuthStore";

export function RutaProtegida() {
  const token = useAuthStore((s) => s.token);

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  return <Outlet />;
}
```

Esta parte	Qué hace
`useAuthStore((s) => s.token)`	Lee el token. Si cambia (login o logout), el componente se vuelve a dibujar
`<Navigate to="/login" replace />`	Sin token: lo mandamos al login
`<Outlet />`	Con token: dibuja la ruta hija que corresponda

Paso 25 · Usar el guard en el mapa de rutas (versión final)

src/App.tsx · ACTUALIZAR

```
import { Navigate, Route, Routes } from "react-router-dom";
import { Layout } from "./components/Layout";
import { RutaProtegida } from "./components/RutaProtegida";
import { LoginPage } from "./pages/LoginPage";
import { NoEncontradaPage } from "./pages/NoEncontradaPage";
import { ProductoDetallePage } from "./pages/ProductoDetallePage";
import { ProductosPage } from "./pages/ProductosPage";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/productos" replace />} />
      <Route path="/login" element={<LoginPage />} />

      <Route element={<RutaProtegida />}>
        <Route element={<Layout />}>
          <Route path="/productos" element={<ProductosPage />} />
          <Route path="/productos/:id" element={<ProductoDetallePage />} />
        </Route>
      </Route>

      <Route path="*" element={<NoEncontradaPage />} />
    </Routes>
  );
}

export default App;
```

Lo único nuevo es la ruta `<Route element={<RutaProtegida />}>` que **envuelve** al Layout. Todo lo que está adentro queda protegido. El login y la 404 quedan afuera: son públicos.

Paso 26 · Cerrar sesión y probar el token en el header

src/components/Layout.tsx · ACTUALIZAR

```
import { NavLink, Outlet, useNavigate } from "react-router-dom";
import { useApi } from "../hooks/useApi";
import { authService } from "../services/auth";
import { useAuthStore } from "../store/useAuthStore";

export function Layout() {
  const navigate = useNavigate();
  const nombre = useAuthStore((s) => s.nombre);
  const cerrarSesion = useAuthStore((s) => s.cerrarSesion);

  // pedimos /auth/me para comprobar que el token funciona
  const { data: yo } = useApi(() => authService.yo());

  const handleLogout = () => {
    cerrarSesion();
    navigate("/login", { replace: true });
  };

  return (
    <div>
      <header>
        <strong>Mi tienda</strong>
        <nav>
          <NavLink to="/productos">Productos</NavLink>
        </nav>
        <span>
          {nombre} {yo && `(${yo.email})`}
        </span>
        <button onClick={handleLogout}>Cerrar sesión</button>
      </header>
      <main>
        <Outlet />
      </main>
    </div>
  );
}
```

Esta parte	Qué hace
`nombre`	Sale del store: lo guardamos en el login
`useApi(() => authService.yo())`	Pide `/auth/me`, una ruta privada. Si ves tu email en el header, **el token se está enviando bien**
`handleLogout`	Evento `onClick`: borra la sesión y vuelve al login

Paso 27 · Cuando el token vence (interceptor de respuesta)

Los tokens tienen fecha de vencimiento. Cuando vence, la API responde **401** a todo. En vez de revisar eso en cada página, agregamos un segundo interceptor que se ejecuta **después de cada respuesta**:

```
src/lib/axios.ts · ACTUALIZAR (versión final)
import axios from "axios";
import { useAuthStore } from "../store/useAuthStore";

export const api = axios.create({
  baseURL: "https://api-funval-g6.onrender.com",
});

// antes de cada peticion: agregar el token
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// despues de cada respuesta: si el token ya no sirve, cerrar sesion
api.interceptors.response.use(
  (respuesta) => respuesta,
  (error) => {
    const esLogin = error.config?.url?.includes("/auth/login");

    if (error.response?.status === 401 && !esLogin) {
      useAuthStore.getState().cerrarSesion();
      window.location.href = "/login";
    }

    return Promise.reject(error);
  },
);
```

Esta parte	Qué hace
`interceptors.response.use(ok, error)`	La primera función se ejecuta si todo salió bien; la segunda, si hubo error
`(respuesta) => respuesta`	Si salió bien, no hacemos nada
`error.response?.status === 401`	La API dice "no s qui n eres": token vencido, borrado o modificado
`esLogin`	El login también responde 401 cuando la contraseña está mal. En ese caso **no** redirigimos: el formulario muestra su propio error
`window.location.href = "/login"`	Aquí no podemos usar `useNavigate` (no es un componente)
`return Promise.reject(error)`	Dejamos pasar el error, para que la página también se entere (su `catch` o el `error` de useApi)

401 vs 403

Código	Significa	Qué hacemos
401	No sé quién eres (no hay token, venció o es falso)	Cerrar sesión e ir al login
403	S qui n **no tienes permiso** (ej: no eres admin)	Mostrar "No tienes acceso". **No** cerrar sesión

Errores comunes

Síntoma	Causa probable
Los datos nunca llegan / tarda mucho la primera vez	La API en Render estaba dormida. Espera ~50 s
`404` en la consola	La ruta está mal escrita (`/product` en vez de `/products/`)
`422` en la consola	Mandaste datos con otro nombre o formato (`nombre` en vez de `name`)
`401` en una ruta privada	No se está mandando el token: revisa el interceptor de request
La página se recarga al enviar el formulario	Falta `e.preventDefault()`
El botón se ejecuta solo al cargar la página	Escribiste `onClick={funcion()}` en vez de `onClick={funcion}`
Ves `[object Promise]` o `undefined`	Falta un `await`
Error "useNavigate() may be used only in the context of a Router"	Falta `<BrowserRouter>` en `main.tsx`
Al recargar pierdes la sesión	Falta `persist` en el store

Resumen: qué hace cada archivo

Archivo	Tema	Responsabilidad
`lib/axios.ts`	Backend + Token	La conexión, y los interceptores que ponen el token y reaccionan al 401
`services/*.ts`	Backend	Una función por cada pedido a la API
`hooks/useApi.ts`	Backend	Manejar cargando / error / datos / reintentar sin repetir código
`components/Buscador.tsx`	Eventos	`onChange` y avisar al padre con una prop `on...`
`pages/LoginPage.tsx`	Eventos + Auth	`onSubmit`, login, guardar sesión y navegar
`main.tsx`	Rutas	Activar el router
`App.tsx`	Rutas	Qué componente va en cada URL y cuáles son privados
`components/Layout.tsx`	Rutas + Auth	Header, menú, `<Outlet />` y cerrar sesión
`store/useAuthStore.ts`	Auth	Guardar el token y compartirlo con toda la app
`components/RutaProtegida.tsx`	Auth	Dejar pasar solo si hay token

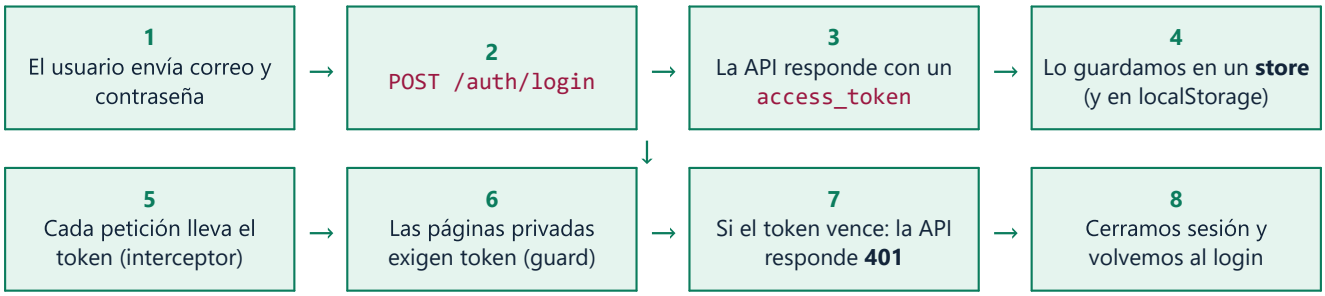
PARTE 4

Autenticación y token

Autenticación es comprobar quien eres (el login). El **token** es la prueba de que ya iniciaste sesión, y lo mandamos en cada petición.

> **Analogía:** en un concierto muestras tu entrada y tu carnet **una sola vez** en la puerta (el login). Te ponen una **pulsera** (el token). Para entrar a cada zona solo muestras la pulsera. La pulsera **vence**: al día siguiente ya no sirve y tienes que volver a la puerta.

El plan



Paso 20 · El servicio de autenticación

src/services/auth.ts · NUEVO

```
import { api } from "../lib/axios";

export interface LoginDatos {
  email: string;
  password: string;
}

interface LoginRespuesta {
  access_token: string;
  token_type: string;
  user_name: string;
  user_role: string;
}

interface Usuario {
  id: number;
  email: string;
  name: string;
  role: string;
}

export const authService = {
  login: async (datos: LoginDatos): Promise<LoginRespuesta> => {
    const respuesta = await api.post<LoginRespuesta>("/auth/login", datos);
    return respuesta.data;
  },

  // ruta privada: solo responde si mandamos un token valido
  yo: async (): Promise<Usuario> => {
    const respuesta = await api.get<Usuario>("/auth/me");
    return respuesta.data;
  },
};
```